

Basic filters

These filters are useful when we need to select an element based on its position in the returned array. We have already seen some ways to do this—for example `get(n)` and `[n]` will give the n^{th} element from the returned set. But `get(n)` or `[n]` can't be used in the chaining of functions. Filters (that we are going to discuss) will return a jQuery object that can be used in chaining, like `$('#button:first').addClass("chained")`.

In our zebra table example, seen in the last article [published in the May 2009 issue of *LF1*] we have given different colours to every other row in a table. The same can be achieved using the odd or even selector. `tr:odd` will select every other odd element on a page, whereas `tr:even` will select every other even element on a page. In our zebra table example, we have used `$('.tr:even').addClass('alt');` to select every even row under the table that has class `tr` and added a class `alt` to them. Table 4 summarises the list of basic filters and their functions with an example.

Basic filters and their functions		
Filter	Function	Examples
:first	To select the first element on the matched set	<code>\$('#button:first')</code> will return the first button element in the page
:last	To select the last element on the matched set	<code>\$('#button:first')</code> will return the last button element in the page
:even	To select the even elements on the matched set	<code>\$('#timetable tr:even')</code> select all the even rows under table with id <code>timetable</code>
:odd	To select the odd elements on the matched set	<code>\$('#timetable tr:odd')</code> select all the odd rows under table with id <code>timetable</code>
:lt(n)	To select the elements that are before the n^{th} position, excluding n	<code>\$('#tr:lt(2)')</code> will return the first 2 elements
:gt(n)	To select the elements that are after the n^{th} position excluding n	<code>\$('#tr:gt(5)')</code> will return all the elements except the first 5
:eq(n)	To select the element in the given n^{th} position	<code>\$('#tr:eq(5)')</code> will return the 5 elements

Table 4

Visibility filters

These filters are used to select elements based on their visibility in the page. An element can be in a visible or hidden state. For example, to select all the visible elements in a page, we can use `$('.visible')`. `$('.hidden')` will select all the hidden elements in a page. If we analyse the previous selector syntaxes, we would know that, by default, all filters will take `('*')` as a matched set when none is explicitly mentioned. Table 5 lists the visibility filters and their functions.

Visibility filters and their functions

Filter	Function	Examples
:visible	All elements that are not selected as part of the :hidden filter	<code>\$('.visible')</code> to select all the elements that are visible
:hidden	All elements that are hidden either by the type or through CSS	<code>\$('.hidden')</code> to select all the elements that are hidden

Table 5

Child filters

These filters are used to select an element based on its position with respect to its containing element, known as the parent. When we need to select the first rows from all the available tables in a page, we can use `$(tr:first-child)`. To make this selector work on a particular element (table), we need to prepend the selector with that element. For example, `$('#tableid tr:first-child')` will select the first row from the table with ID `tableid`. As we saw in basic filters, here too we have the n^{th} child (number/odd/even). For example, `$('#tableid tr:nth-child(odd)')` will select all the odd columns from the page.

We can use the `:only-child` selector to select the element that is the ONLY child for its parent.

```
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en">
<div></div>
<div>
  <div><span>MySpan1</span><button>MyButton1</button></div>
  <div><span>MySpan2</span></div>
</div>
</html>
```

In the above HTML snippet, `$(span:only-child)` will select the span with value `MySpan2`. Table 6 gives the complete list of child selectors with their usage patterns.

Child selectors and their usage patterns

Filter	Function	Examples
parent>child	All children of the parent	<code>\$(table>tr)</code> , matches all the table rows
:first-child	To select any element that is the first child for its parent	<code>\$('#tableid tr:first-child')</code> , matches the first row from the table with ID <code>tableid</code>
:last-child	To select all the elements that are the last children for its parents	<code>\$('#tableid tr:last-child')</code> , matches all that's in the last row from the table with ID <code>tableid</code>
:nth-child(n/odd/even)	To select a particular row using its position within the parent. Can be group-based and be odd or even	<code>\$(table tr:odd)</code> to select all the odd columns from all the available tables

Table 6